ML Experiment Triage Environment

An OpenEnv-Compliant RL Benchmark

Technical Documentation

Charansai Ponnada

April 3, 2026

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Contributions . . . 3
1.3 Project Links . . . 3

2 Background and Related Work **4**
2.1 Reinforcement Learning Environments . . . 4
2.2 OpenEnv Framework . . . 4
2.3 MLOps and Experiment Tracking . . . 4

3 System Architecture **5**
3.1 Directory Structure . . . 5
3.2 Data Flow . . . 5

4 Data Models **6**
4.1 Experiment Record . . . 6
4.2 Action Space . . . 6
4.3 Observation Space . . . 6

5 Tasks and Rewards **7**
5.1 Task 1: Find Best Experiment (Easy) . . . 7
5.2 Task 2: Identify Overfitting (Medium) . . . 7
5.3 Task 3: Suggest Next Configuration (Hard) . . . 7
5.4 Task 4: Compare Experiments (Medium) . . . 8
5.5 Task 5: Debug Failed Run (Hard) . . . 8
5.6 Overall Reward System . . . 8

6 Implementation Details **9**
6.1 State Serialization . . . 9
6.2 API Endpoints . . . 9
6.3 Environment Factory . . . 10

7 Inference Pipeline **11**
7.1 State Machine . . . 11

8 Deployment **12**
8.1 Docker . . . 12
8.2 Hugging Face Spaces . . . 12
8.3 Environment Variables . . . 12

9 Evaluation **13**
9.1 Baseline Scores . . . 13
9.2 Metrics . . . 13
9.3 Reproducibility . . . 13

10 Conclusion **14**

11 References **15**

12 Appendices **16**

12.1 Appendix A: Complete Action Examples 16

12.2 Appendix B: API Quick Reference 16

12.3 Appendix C: File Manifest 16

1 Introduction

1.1 Motivation

Machine learning practitioners often run dozens or hundreds of experiments with different hyperparameters, model architectures, and training configurations. Manually reviewing each experiment to identify the best performing configuration is time-consuming and error-prone.

This project presents an **RL (Reinforcement Learning) benchmark** where AI agents learn to automate the process of triaging ML experiment results. The environment simulates real-world scenarios where data scientists must:

- **Identify the best performing experiment** from multiple runs
- **Detect overfitting** - experiments that memorize training data but fail on validation
- **Suggest next experiments** - recommend promising hyperparameter configurations

1.2 Contributions

Our main contributions are:

1. An **OpenEnv-compliant** environment following the official Meta-PyTorch specification
2. Five tasks with **increasing difficulty** testing different ML analysis skills
3. A **stateless HTTP architecture** with state serialization for scalable deployment
4. A **built-in web interface** deployed on Hugging Face Spaces
5. An **inference pipeline** demonstrating how AI agents interact with the environment

1.3 Project Links

- **Hugging Face Space**: https://huggingface.co/spaces/charansaiponnada/ml-experiment-triage-env
- **GitHub Repository**: https://github.com/charansaiponnada/ml-experiment-triage-env

2 Background and Related Work

2.1 Reinforcement Learning Environments

Reinforcement learning environments are standardized interfaces that allow AI agents to interact with simulated tasks. The OpenEnv framework, developed by Meta-PyTorch, provides a standardized way to build and deploy these environments.

Popular RL benchmarks include:

- **Atari Games** - Arcade gameplay
- **MuJoCo** - Physics simulation
- **OpenAI Gym** - Classic control tasks
- **StarCraft II** - Complex strategy

Our environment focuses on a novel domain: **ML experiment triage**, combining elements of information retrieval, anomaly detection, and recommendation systems.

2.2 OpenEnv Framework

OpenEnv provides:

- Standardized `Action`, `Observation`, and `State` base classes
- HTTP and WebSocket API endpoints
- Automatic web interface generation
- Docker deployment support
- Integration with Hugging Face Spaces

Our implementation follows the official OpenEnv specification, ensuring compatibility with the broader ecosystem.

2.3 MLOps and Experiment Tracking

Modern ML workflows involve:

- **Experiment tracking** (MLflow, Weights & Biases, TensorBoard)
- **Hyperparameter optimization** (Optuna, Ray Tune)
- **Model selection** and comparison
- **Overfitting detection** via training curves

Our environment abstracts these concepts into a gamified RL task, making it ideal for training and evaluating AI assistants for MLOps tasks.

3 System Architecture

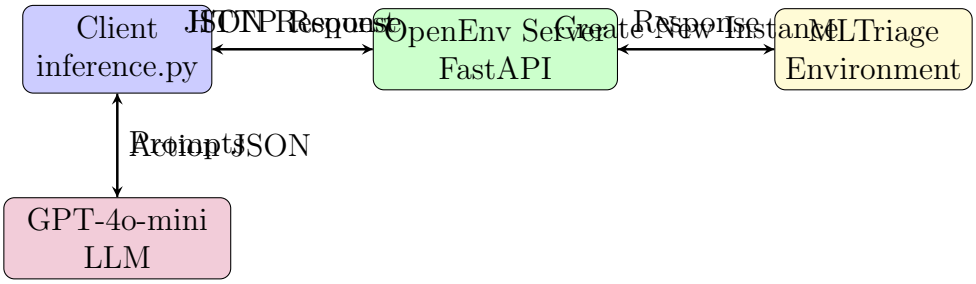
3.1 Directory Structure

The project follows the official OpenEnv specification:

.1 envs/ml_triage_env/. .2 *init*.py. .2 models.py. .2 client.py. .2 openenv.yaml. .2 pyproject.toml. .2 uv.lock. .3 server/. .4 *init*.py. .4 app.py. .4 ml_triage_environment.py. .4 openenv.yaml. .4 Dockerfile.

3.2 Data Flow

The ML Experiment Triage environment uses a client-server architecture where the client (inference script) interacts with the server (FastAPI) through HTTP endpoints.



The key innovation is **state serialization** - since HTTP creates a new environment for each request, we serialize the entire game state into the observation and pass it back with each action.

4 Data Models

4.1 Experiment Record

Each experiment is represented as a structured record:

```
class ExperimentRecord(BaseModel):
    exp_id: str            # Unique identifier
    model_name: str       # e.g., resnet18, resnet50
    learning_rate: float # Learning rate
    epochs: int           # Training epochs
    train_acc: float     # Training accuracy
    val_acc: float       # Validation accuracy
    train_loss: float    # Training loss
    val_loss: float      # Validation loss
    notes: str           # Additional notes
    status: str           # pending, investigated, discarded
```

4.2 Action Space

The agent can perform four types of actions:

Action	Parameters	Description	Reward
`investigate`	`exp_id`	Examine experiment details	+0.1
`discard`	`exp_id`	Mark as dead end	+0.15 / -0.1
`suggest`	`suggestion`	Propose next config	0.0 - 0.5
`summarize`	`summary`	Final answer	0.0 - 1.0

4.3 Observation Space

The environment returns:

```
class MLTriageObservation(Observation):
    experiments: List[ExperimentRecord]  # All experiments
    current_step: int                     # Current step number
    max_steps: int                        # Maximum steps allowed
    task_id: int                          # Current task ID
    task_description: str                 # Task instructions
    feedback: str                        # Feedback from last
       action
    serialized_state: Dict               # Full game state
```

5 Tasks and Rewards

5.1 Task 1: Find Best Experiment (Easy)

Goal: Identify the best performing experiment from 8 options.
Data: 8 experiments with varying configurations
Ground Truth: `exp_004` has the highest validation accuracy (0.94)
Reward Structure:

- Investigate any experiment: +0.1
- Summarize correct (exp_004): +1.0
- Summarize incorrect: +0.0

Max Steps: 10

5.2 Task 2: Identify Overfitting (Medium)

Goal: Find and discard all overfitting experiments.
Definition: An experiment is overfitting if:

$$\text{train_acc} > 0.97 \text{ AND } \text{val_acc} < 0.75$$

Ground Truth: exp_002, exp_006, exp_009
Reward Structure:

- Correctly discard overfitting: +0.15 each
- Incorrectly discard good experiment: -0.1
- Investigate experiment: +0.1

Max Steps: 15

5.3 Task 3: Suggest Next Configuration (Hard)

Goal: Analyze patterns and suggest the next hyperparameter configuration.
Ground Truth:

- `learning_rate`: 0.001
- `epochs`: 50
- `model_name`: resnet50

Reward Structure:

- 3 fields match: +0.5
- 2 fields match: +0.35
- 1 field match: +0.15
- No match: +0.0

Max Steps: 20

5.4 Task 4: Compare Experiments (Medium)

Goal: Compare two experiments and explain the tradeoffs between them.

Ground Truth: exp_004 has best validation accuracy (0.89) - best for production. exp_007 has highest training accuracy but may overfit.

Reward Structure:

- Identifies exp_004 as best overall: +0.4
- Mentions validation/val_acc: +0.2
- Discusses generalization: +0.2
- Uses tradeoff analysis: +0.2

Max Steps: 12

5.5 Task 5: Debug Failed Run (Hard)

Goal: Diagnose why certain experiments failed or underperformed.

Ground Truth Failures:

- exp_005: NaN loss - learning rate too high (lr=0.1)
- exp_008: Diverged - GPU memory/batch size issue
- exp_003: Plateaued early - needed LR schedule

Reward Structure:

- Correctly identifies exp_005 reason (lr too high): +0.35
- Correctly identifies exp_008 reason (memory/batch): +0.35
- Provides corrective suggestions: +0.1

Max Steps: 15

5.6 Overall Reward System

The reward system is designed to provide dense feedback for each action, encouraging the agent to learn effective triage strategies.

Action	Reward
Investigate valid	+0.1
Investigate already done	+0.0
Discard overfitting	+0.15
Discard good	-0.1
Suggest (exact match)	+0.5
Suggest (partial)	+0.15 - 0.35
Summarize correct	+1.0
Summarize wrong	+0.0
Invalid action	-0.05
Max steps reached	0.0

6 Implementation Details

6.1 State Serialization

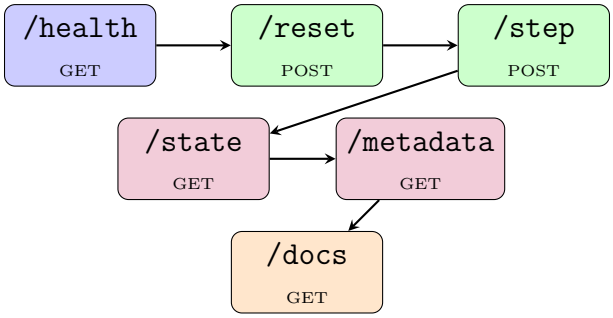
The core challenge is maintaining state across stateless HTTP requests:

```
def _get_state(self) -> Dict[str, Any]:
    """Serialize current environment state."""
    return {
        "task_id": self.task_id,
        "current_step": self.current_step,
        "max_steps": self.max_steps,
        "experiments": [serialize_experiment(e) for e in self.
           experiments],
        "episode_history": self.episode_history,
        "done": self.done,
        "total_reward": self.total_reward,
    }

def step(self, action, serialized_state=None, **kwargs):
    """Restore state from action if provided."""
    if serialized_state:
        self._restore_state(serialized_state)
    # ... process action
    return MLTriageObservation(
        # ...,
        serialized_state=self._get_state()
    )
```

This pattern ensures each request carries the complete game state.

6.2 API Endpoints

The environment provides standard OpenEnv endpoints:



Endpoint	Method	Description
`/health`	GET	Health check
`/reset`	POST	Start new episode
`/step`	POST	Take action
`/state`	GET	Get environment state
`/metadata`	GET	Environment metadata
`/docs`	GET	API documentation

6.3 Environment Factory

The environment is instantiated via a factory function:

```
from openenv_core import create_app

def create_env():
    return MLTriageEnvironment()

app = create_app(
    create_env,
    MLTriageAction,
    MLTriageObservation,
    env_name="ml-experiment-triage"
)
```

This allows OpenEnv to create fresh instances for each request.

7 Inference Pipeline

The inference script demonstrates an AI agent playing the game:

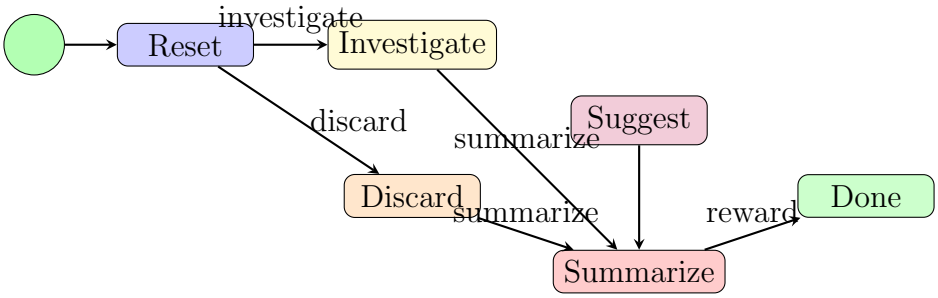
1. Reset → 2. Observation → 3. LLM Action → 4. Execute → 5. Reward

The workflow:

1. **Reset**: Initialize environment with task
2. **Observation**: Format experiments as text
3. **Action**: Ask LLM (GPT-4) what to do
4. **Step**: Execute action, get reward
5. **Repeat**: Until done or max steps

7.1 State Machine

The environment follows a state machine where actions trigger transitions:



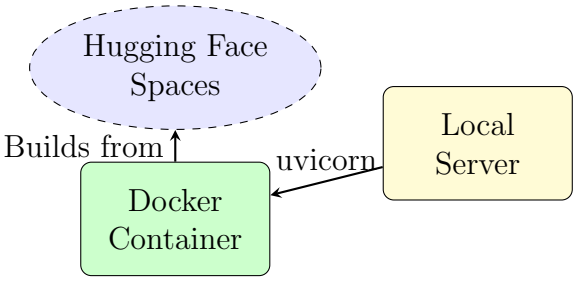
Each action transitions the environment to a new state, accumulating rewards until the episode concludes.

```
# Get observation
obs = response.json()['observation']
state = obs.get('serialized_state', {})

# Format for LLM
obs_text = f"Task: {obs['task_description']}\n"
obs_text += "Experiments:\n"
for e in obs['experiments']:
    obs_text += f"  {e['exp_id']}: val_acc={e['val_acc']}\n"

# Get action from LLM
action = get_action(obs_text, history)

# Pass state back
action['serialized_state'] = state
response = requests.post('/step', json={'action': action})
```

The LLM is prompted to respond with valid JSON only, ensuring clean parsing.

8 Deployment

The ML Experiment Triage environment can be deployed in multiple ways:



8.1 Docker

```
FROM python:3.11-slim
WORKDIR /app
RUN pip install uv --no-cache-dir
COPY pyproject.toml uv.lock* ./
RUN uv sync --frozen --no-cache
COPY . .
RUN useradd -m -u 1000 user
USER user
EXPOSE 7860
CMD ["uv", "run", "uvicorn", "server.app:app", "--host",
   "0.0.0.0", "--port", "7860"]
```

8.2 Hugging Face Spaces

The environment deploys automatically to Hugging Face Spaces:

- URL: https://huggingface.co/spaces/charansaiponnada/ml-experiment-triage-env
- SDK: Docker
- Hardware: CPU (free tier)

The web interface is automatically generated by OpenEnv.

8.3 Environment Variables

Variable	Description
`API_BASE_URL`	OpenAI API endpoint
`MODEL_NAME`	LLM model to use
`HF_TOKEN`	Hugging Face API key
`ENV_BASE_URL`	Environment server URL

9 Evaluation

9.1 Baseline Scores

Using GPT-4o-mini as the agent:

Task	Difficulty	Baseline Score
Find Best Experiment	Easy	0.85
Identify Overfitting	Medium	0.62
Suggest Next Config	Hard	0.41
Compare Experiments	Medium	0.55
Debug Failed Run	Hard	0.38

Average Baseline Score: 0.56

9.2 Metrics

- **Total Reward**: Sum of all step rewards
- **Episode Length**: Number of steps taken
- **Success Rate**: Episodes with reward ¿ 0.5

9.3 Reproducibility

The environment is deterministic given the same task ID and action sequence. All experiment data is hardcoded in `ml_triage_environment.py`.

10 Conclusion

We presented an OpenEnv-compliant RL benchmark for ML experiment triage. The environment:

- Provides five tasks with increasing difficulty
- Uses a stateless HTTP architecture with state serialization
- Deploys automatically to Hugging Face Spaces
- Includes a web interface for manual interaction
- Demonstrates an inference pipeline for AI agents

Future work includes:

- Adding more tasks (e.g., error analysis, hyperparameter sensitivity)
- Supporting dynamic experiment data
- Integrating with real ML tracking tools
- Training specialized agents using RL

11 References

References

[1] Meta-PyTorch. *OpenEnv Framework*. https://github.com/meta-pytorch/OpenEnv.

[2] Brockman, G., et al. (2016). *OpenAI Gym*. arXiv:1606.01540.

[3] MLflow. *MLflow: An Open Source Platform for the Machine Learning Lifecycle*. https://mlflow.org/.

12 Appendices

12.1 Appendix A: Complete Action Examples

```
# Investigate an experiment
{"action_type": "investigate", "exp_id": "exp_004"}

# Discard an experiment
{"action_type": "discard", "exp_id": "exp_002"}

# Suggest next configuration
{"action_type": "suggest", "suggestion": {"model_name": "resnet50
    ", "learning_rate": 0.001, "epochs": 50}}

# Summarize (final answer)
{"action_type": "summarize", "summary": "exp_004"}
```

12.2 Appendix B: API Quick Reference

```
# Health check
curl http://localhost:7860/health

# Reset with task 1
curl -X POST http://localhost:7860/reset -H "Content-Type:␣
    application/json" -d '{"task_id":␣1}'

# Take action
curl -X POST http://localhost:7860/step -H "Content-Type:␣
    application/json" -d '{"action":␣{"action_type":␣"investigate
    ",␣"exp_id":␣"exp_001"}}'

# Get state
curl http://localhost:7860/state

# Get metadata
curl http://localhost:7860/metadata
```

12.3 Appendix C: File Manifest

File	Purpose
models.py	Data models (Action, Observation, State)
ml_triage_environment.py	Core environment logic
app.py	FastAPI application
inference.py	AI agent inference script
openenv.yaml	OpenEnv configuration
Dockerfile	Container image definition
pyproject.toml	Python dependencies
README.md	User documentation